

NeuralNova — AI-Based Network Intrusion and Cyber Threat Detection System



Submitted by

Sheeza Eman (S23BINFT1M01176)

Submitted to

Dr. Muhammad Asadullah

Department of Information Technology Faculty of Computing
The Islamia University of Bahawalpur

Meeting Details

Sr No	Details	Date	Supervisor Signature
1			
2			
3			

Summary

NeuralNova is an advanced AI-powered cybersecurity system designed to detect, classify, and respond to network intrusions, malware threats, and phishing attacks in real time. This Software Design Document (SDD) describes the architectural design, component specifications, database schema, interface definitions, and deployment model of the NeuralNova system. The system employs a hybrid machine learning architecture combining Random Forest classification and Long Short-Term Memory (LSTM) deep learning networks to deliver proactive threat detection beyond the capabilities of conventional signature-based tools. Developed using Python, TensorFlow, FastAPI, React.js, and MongoDB, NeuralNova is architected as a scalable, modular, and maintainable cybersecurity platform suitable for enterprise environments, academic institutions, and government networks.

Table of Contents

- 1 Introduction 5**
 - 1.1 Purpose 5
 - 1.2 Scope 5
 - 1.3 Design Goals and Constraints 5
 - 1.4 Definitions and Acronyms 6
- 2 System Architecture 7**
 - 2.1 Architectural Overview 7
 - 2.2 Layered Architecture 7
 - 2.3 System Modules 7
 - 2.4 Technology Stack 8
- 3 Component Design 9**
 - 3.1 Network Packet Capture Module 9
 - 3.2 Feature Extraction Module 9
 - 3.3 AI Threat Detection Engine 10
 - 3.4 Malware Analysis Module 10
 - 3.5 Phishing Detection Module 10
 - 3.6 FastAPI REST Backend 11
 - 3.7 React.js Monitoring Dashboard 11
 - 3.8 MongoDB Data Layer 11
- 4 Database Design 12**
 - 4.1 Data Model Overview 12
 - 4.2 Collections and Schema 12
- 5 Interface Design 13**
 - 5.1 API Endpoints 13
 - 5.2 External API Integrations 14
- 6 Security Design 14**
- 7 Deployment Architecture 15**
- 8 References 16**

1. Introduction

NeuralNova is an Artificial Intelligence (AI)-based cybersecurity application designed to improve network security by monitoring, detecting, and classifying cyber threats in real time. This Software Design Document (SDD) translates the requirements outlined in the Software Requirements Specification (SRS) into a concrete technical blueprint for implementation. The document covers the system architecture, component design, data design, interface specifications, and security considerations.

1.1 Purpose

The purpose of this SDD is to provide a detailed technical design of the NeuralNova system to guide developers during implementation. It bridges the gap between the requirements defined in the SRS and the actual source code, ensuring a consistent, modular, and maintainable system.

1.2 Scope

This document covers the complete technical design of all NeuralNova modules: network packet capture, feature extraction, AI-based threat detection (Random Forest + LSTM), malware scanning, phishing detection, FastAPI backend, React.js dashboard, and MongoDB data storage. External components such as the VirusTotal API and PhishTank API are described from an integration perspective only.

1.3 Design Goals and Constraints

Goal / Constraint	Description
Real-Time Performance	Threat classification must complete within 1 second per packet
Accuracy	Minimum 95% intrusion detection accuracy on benchmark datasets
Modularity	Each component independently testable and replaceable
Scalability	System must handle up to 1,000 packets per second
Security	API keys stored in .env files; never exposed in code or repositories
Compatibility	Runs on Windows 10/11 with Python 3.10 and standard web browser
Portability	Designed for single-host deployment; extendable to distributed architecture

1.4 Definitions and Acronyms

Term	Definition
AI	Artificial Intelligence
LSTM	Long Short-Term Memory neural network
RF	Random Forest classifier
DoS	Denial of Service attack
DDoS	Distributed Denial of Service attack
API	Application Programming Interface
REST	Representational State Transfer
SRS	Software Requirements Specification

Term	Definition
SDD	Software Design Document
ML	Machine Learning
NLP	Natural Language Processing
PCAP	Packet Capture
JSON	JavaScript Object Notation
MongoDB	NoSQL document-oriented database

2. System Architecture

2.1 Architectural Overview

NeuralNova follows a layered pipeline architecture where each layer performs a distinct processing function and passes its output to the next layer. The design separates concerns between data capture, AI inference, backend logic, external integrations, and frontend presentation. This separation ensures modularity, independent testability, and ease of maintenance.

2.2 Layered Architecture

Layer	Component	Responsibility
1	Layer 1 — Data Acquisition	Scapy / PyShark capture live network packets from the selected network interface.
2	Layer 2 — Feature Engineering	Pandas and NumPy transform raw packets into numerical feature vectors suitable for ML inference.
3	Layer 3 — AI Inference Engine	Random Forest performs initial threat classification; LSTM performs sequential pattern validation.
4	Layer 4 — Threat Intelligence	VirusTotal API scans malware hashes; PhishTank API + NLP verifies phishing URLs.
5	Layer 5 — Backend API	FastAPI serves AI model results, manages MongoDB writes, and triggers automated responses.
6	Layer 6 — Presentation Layer	React.js dashboard renders live charts, threat tables, and alert notifications.
7	Layer 7 — Persistence Layer	MongoDB stores all threat logs, scan results, and system events.

2.3 System Modules

The system is decomposed into eight primary modules, each responsible for a specific aspect of the cybersecurity pipeline:

Module ID	Module Name	Primary Technology
MOD-01	Network Packet Capture Module	Scapy, PyShark
MOD-02	Feature Extraction Module	Pandas, NumPy
MOD-03	AI Threat Detection Engine	Scikit-learn (RF), TensorFlow/Keras (LSTM), XGBoost
MOD-04	Malware Analysis Module	VirusTotal API, hashlib
MOD-05	Phishing Detection Module	PhishTank API, spaCy / NLTK (NLP fallback)
MOD-06	FastAPI REST Backend	FastAPI, Uvicorn, Pydantic
MOD-07	React.js Monitoring Dashboard	React.js, Chart.js, Axios
MOD-08	MongoDB Data Layer	MongoDB, PyMongo

2.4 Technology Stack

Category	Technology	Version / Notes
Language	Python	3.10+
Machine Learning	Scikit-learn, XGBoost	Latest stable
Deep Learning	TensorFlow, Keras	2.x
Network Capture	Scapy, PyShark	Latest stable
Backend Framework	FastAPI, Uvicorn	0.100+
Frontend	React.js, Chart.js, Axios	18.x / 4.x
Database	MongoDB, PyMongo	6.x / 4.x
Threat Intel	VirusTotal API, PhishTank API	v3 / v2
Training Datasets	CICIDS2017, KDD Cup 99	Benchmark datasets
DevOps	Git, GitHub, VS Code	Latest

3. Component Design

This section details the internal design of each NeuralNova module, including inputs, outputs, processing logic, and inter-module communication.

3.1 Network Packet Capture Module (MOD-01)

This module interfaces directly with the host network adapter to capture raw network packets in real time using Scapy and PyShark. It implements a configurable sniffing loop that continuously reads packets from the selected interface and forwards them to MOD-02 via an in-memory packet queue.

Attribute	Detail
Input	Raw network interface stream (Ethernet frames, IP packets, TCP/UDP segments)
Output	Parsed packet objects (source IP, destination IP, protocol, payload, timestamp)
Key Classes	PacketSniffer, InterfaceSelector, PacketQueue
Libraries	scapy.all, pyshark
Threading	Runs in a dedicated background thread to avoid blocking the API
Error Handling	Catches PermissionError (root required), interface-not-found, malformed packets

3.2 Feature Extraction Module (MOD-02)

This module consumes raw packet objects from MOD-01 and transforms them into structured numerical feature vectors that can be fed directly into the ML models in MOD-03. Feature engineering follows the CICIDS2017 feature set standard.

Attribute	Detail
Input	Parsed packet objects from MOD-01
Output	Feature vector: 78-dimensional NumPy array per packet flow
Key Features	Flow duration, packet length stats, IAT mean/std, byte counts, protocol flags, port numbers
Key Classes	FeatureExtractor, FlowAggregator, FeatureNormalizer
Libraries	pandas, numpy, scikit-learn (StandardScaler)
Normalization	Min-max normalization applied before inference; scaler saved as scaler.pkl

3.3 AI Threat Detection Engine (MOD-03)

This is the core intelligence module of NeuralNova. It implements a dual-model hybrid approach: Random Forest for initial per-packet classification and LSTM for sequential pattern analysis across time windows. The outputs are combined using a confidence-weighted ensemble strategy.

Attribute	Detail
Input	Feature vector (78 dims) from MOD-02; sequence of 50 vectors for LSTM
Output	Threat label: Normal / DoS / DDoS / Port Scan / Brute Force / Malware; confidence score (0–1)

Attribute	Detail
Random Forest	100 estimators, max_depth=20, trained on CICIDS2017; handles structured tabular features
LSTM Architecture	2 LSTM layers (128 units each) + Dense(64) + Dropout(0.3) + Softmax output; sequence length=50
XGBoost	Used as validation booster for borderline RF predictions (confidence 0.4–0.6)
Ensemble Logic	If RF confidence ≥ 0.7 : use RF result. If < 0.7 : run LSTM for validation
Model Files	rf_model.pkl, lstm_model.h5, xgb_model.pkl, scaler.pkl (loaded at startup)
Training Datasets	CICIDS2017 (primary), KDD Cup 99 (supplementary)

3.4 Malware Analysis Module (MOD-04)

Provides on-demand file-based malware scanning by integrating with the VirusTotal API v3. File hashes are computed locally and submitted to VirusTotal; the module returns a structured scan report indicating the detection verdict from 70+ antivirus engines.

Attribute	Detail
Input	File uploaded by user via REST endpoint (/api/scan/file)
Output	JSON report: verdict (Malicious / Clean / Suspicious), engine hits, file hash
Hash Algorithm	SHA-256 (primary), MD5 (fallback for VirusTotal lookup)
API Rate Limit	Free tier: 4 requests/minute; handled with exponential back-off retry
Caching	Hash-to-result cache in MongoDB to avoid duplicate API calls
Error Handling	API key invalid, file size > 32 MB, API quota exceeded → error response with code

3.5 Phishing Detection Module (MOD-05)

Verifies URLs submitted by the user against the PhishTank community database. If the URL is not found in PhishTank, a local NLP-based classifier analyses lexical and structural URL features to produce a phishing probability score.

Attribute	Detail
Input	URL string submitted via REST endpoint (/api/scan/url)
Output	Verdict: Phishing / Safe; confidence score (0–1); source: PhishTank or NLP
Primary Method	PhishTank API lookup — returns known phishing status from crowd-sourced DB
Fallback Method	NLP classifier: extracts URL length, TLD, subdomain count, suspicious keywords, entropy
NLP Model	Logistic Regression trained on PhishTank + Tranco URL dataset; F1 > 0.92
Error Handling	Invalid URL format → 400 error; API timeout → fallback to NLP; rate limit → queue

3.6 FastAPI REST Backend (MOD-06)

The FastAPI backend serves as the central orchestrator of all NeuralNova components. It exposes a RESTful API consumed by the React.js dashboard, manages the lifecycle of all modules, handles authentication, and writes all events to MongoDB.

Attribute	Detail
Framework	FastAPI with Uvicorn ASGI server
Key Responsibilities	Start/stop monitoring, serve model predictions, proxy external API calls, write MongoDB logs, trigger IP blocking
Authentication	JWT-based token auth for dashboard API calls; API keys in .env
CORS	CORS middleware enabled for localhost:3000 (React dev server)
Background Tasks	Packet capture runs as FastAPI BackgroundTask; results pushed via WebSocket
Response Format	All responses return JSON with fields: status, data, timestamp, error_code

3.7 React.js Monitoring Dashboard (MOD-07)

The dashboard provides a live visual interface for security analysts. It polls the FastAPI backend every 5 seconds to update charts, threat tables, and alert panels.

Attribute	Detail
Framework	React.js 18 with functional components and hooks
Charts	Chart.js for live traffic volume, threat type distribution, timeline charts
HTTP Client	Axios for REST calls; native WebSocket for real-time push alerts
Key Views	Dashboard Overview, Live Threat Table, Malware Scanner, Phishing Checker, Threat History
Refresh Rate	Auto-refresh every 5 seconds; WebSocket push for critical alerts
Alert Mechanism	Browser notification + in-app toast alert for HIGH severity threats

3.8 MongoDB Data Layer (MOD-08)

MongoDB acts as the persistent storage layer for all NeuralNova threat events, scan results, and system logs. PyMongo is used as the database driver within the FastAPI backend.

Attribute	Detail
Database	MongoDB 6.x; database name: neuralnova_db
Driver	PyMongo 4.x
Connection	Connection string in .env: MONGO_URI=mongodb://localhost:27017
Collections	threats, scans, urls, system_logs, blocked_ips
Indexing	timestamp (descending) index on threats and scans for fast dashboard queries
Data Retention	Configurable TTL index on threats collection (default: 90 days)

4. Database Design

4.1 Data Model Overview

NeuralNova uses MongoDB as a schema-flexible NoSQL document store. All collections store documents in BSON format. The primary collections are described below with their field definitions, data types, and example values.

4.2 Collections and Schema

Collection: threats

Field	Type	Description
_id	ObjectId	MongoDB auto-generated unique identifier
timestamp	DateTime	UTC datetime when threat was detected
threat_type	String	DoS DDoS Port Scan Brute Force Malware Normal
source_ip	String	Source IP address of the detected traffic
dest_ip	String	Destination IP address
confidence	Float	Model confidence score (0.0 – 1.0)
model_used	String	RF LSTM Ensemble
action_taken	String	alert_sent ip_blocked logged_only
severity	String	LOW MEDIUM HIGH CRITICAL
raw_features	Object	Extracted feature vector stored as key-value pairs

Collection: scans

Field	Type	Description
_id	ObjectId	MongoDB auto-generated identifier
timestamp	DateTime	UTC datetime of the scan request
file_name	String	Original uploaded filename
sha256	String	SHA-256 hash of the uploaded file
verdict	String	Malicious Clean Suspicious
engine_hits	Integer	Number of AV engines flagging as malicious (VirusTotal)
total_engines	Integer	Total AV engines queried
vt_report_url	String	Full VirusTotal report URL

Collection: urls

Field	Type	Description
_id	ObjectId	MongoDB auto-generated identifier
timestamp	DateTime	UTC datetime of the URL check

Field	Type	Description
url	String	Submitted URL string
verdict	String	Phishing Safe
confidence	Float	Confidence score from NLP model (0.0 – 1.0)
source	String	PhishTank NLP_Fallback

Collection: blocked_ips

Field	Type	Description
_id	ObjectId	MongoDB auto-generated identifier
ip_address	String	Blocked IP address
blocked_at	DateTime	UTC datetime when block was applied
reason	String	Associated threat_type that triggered the block
threat_id	ObjectId	Reference to the triggering document in threats collection
is_active	Boolean	True if block is currently active; False if lifted

5. Interface Design

5.1 API Endpoints

The FastAPI backend exposes the following RESTful endpoints. All endpoints return JSON responses with HTTP status codes conforming to REST conventions.

Method	Endpoint	Description	Auth
POST	/api/monitoring/start	Start packet capture and AI monitoring	JWT
POST	/api/monitoring/stop	Stop packet capture and enter standby	JWT
GET	/api/threats	Retrieve latest threat log entries (paginated)	JWT
GET	/api/threats/{id}	Retrieve single threat record by ID	JWT
GET	/api/stats	Get aggregated threat statistics for dashboard	JWT
POST	/api/scan/file	Upload file for VirusTotal malware scan	JWT
POST	/api/scan/url	Submit URL for phishing detection	JWT
GET	/api/blocked-ips	Retrieve list of all currently blocked IPs	JWT
DELETE	/api/blocked-ips/{ip}	Remove an IP address from the block list	JWT
POST	/api/auth/login	Authenticate user and receive JWT token	None
WS	/ws/alerts	WebSocket channel for real-time push alerts	Token

5.2 External API Integrations

Service	API Version	Usage	Auth Method
VirusTotal	v3	File hash lookup; malware scan	x-apikey header
PhishTank	v2	URL phishing lookup	app_key query param

All API keys are stored in a .env file at the project root and loaded via python-dotenv. Keys are never committed to version control. The .gitignore file excludes .env explicitly.

6. Security Design

Security is a first-class design concern in NeuralNova. The following security controls are designed into the system at the component and infrastructure level:

Control	Implementation
Authentication	JWT tokens issued on login; tokens expire after 8 hours; refresh token mechanism
API Key Management	All external API keys stored in .env; loaded with python-dotenv; never in source code or GitHub
Input Validation	Pydantic models validate all API request bodies; URL format checked via urllib.parse; file type whitelist enforced
IP Auto-Block	Malicious source IPs are auto-blocked using OS-level firewall rules (iptables on Linux / Windows Firewall on Windows)
Sensitive Data	No user PII is stored; all logged data relates to network flows and file hashes only
Dependency Security	requirements.txt pins all versions; pip audit run in CI pipeline to detect known CVEs
CORS Policy	CORS restricted to localhost:3000 in development; configurable whitelist in production
Rate Limiting	FastAPI slowapi middleware applies rate limits: 100 req/min per IP on scan endpoints

7. Deployment Architecture

NeuralNova is designed as a single-host deployment for the current version. All components run on the same machine. The system can be extended to a distributed deployment in future iterations.

Component	Host Process	Port / Interface
FastAPI Backend	uvicorn main:app --reload	HTTP: 8000
React.js Frontend	npm start	HTTP: 3000
MongoDB	mongod	TCP: 27017
Packet Capture	BackgroundTask in FastAPI	Network interface (promiscuous mode)
WebSocket Server	Integrated in FastAPI	WS: 8000/ws/alerts

7.1 Environment Setup

The following steps describe the deployment and setup procedure:

1. Install Python 3.10+, Node.js 18+, and MongoDB 6.x on the target machine.
2. Clone the repository: `git clone https://github.com/[username]/NeuralNova.git`
3. Install Python dependencies: `pip install -r requirements.txt`
4. Install frontend dependencies: `cd frontend && npm install`
5. Configure environment variables: copy `.env.example` to `.env` and fill in API keys.
6. Download or train ML models and place model files in `/models/` directory.
7. Start MongoDB service: `mongod --dbpath ./data`
8. Start FastAPI backend: `uvicorn main:app --host 0.0.0.0 --port 8000`
9. Start React frontend: `npm start` (from `/frontend` directory)
10. Open browser at `http://localhost:3000` to access the monitoring dashboard.

7.2 Directory Structure

```
NeuralNova/  
+-- main.py # FastAPI application entry point  
+-- requirements.txt # Python dependencies  
+-- .env.example # Environment variable template  
+-- models/ # Trained ML model files (.pkl, .h5)  
+-- modules/  
| +-- capture.py # MOD-01: Packet capture  
| +-- features.py # MOD-02: Feature extraction  
| +-- detector.py # MOD-03: AI detection engine  
| +-- malware.py # MOD-04: VirusTotal integration  
| +-- phishing.py # MOD-05: PhishTank + NLP  
| +-- database.py # MOD-08: MongoDB operations  
+-- routers/  
| +-- monitoring.py # /api/monitoring endpoints  
| +-- scans.py # /api/scan endpoints  
| +-- auth.py # /api/auth endpoints  
+-- frontend/ # React.js application  
| +-- src/  
| | +-- components/ # React components  
| | +-- pages/ # Dashboard, Scanner, History  
| | +-- App.js
```



```
| +-- package.json  
+-- data/ # MongoDB data directory
```

8. References

1. Sharafaldin, I., Lashkari, A. H., & Ghorbani, A. A. (2018). Toward Generating a New Intrusion Detection Dataset. ICISSP 2018.
2. KDD Cup 1999 Dataset — UCI Machine Learning Repository. <http://kdd.ics.uci.edu/databases/kddcup99/>
3. VirusTotal API Documentation. <https://developers.virustotal.com/reference>
4. PhishTank Developer Information. https://www.phishtank.com/developer_info.php
5. TensorFlow Documentation. <https://www.tensorflow.org/guide>
6. FastAPI Documentation. <https://fastapi.tiangolo.com>
7. Scikit-learn Documentation. <https://scikit-learn.org/stable/documentation.html>
8. React.js Official Documentation. <https://react.dev>
9. MongoDB Documentation. <https://www.mongodb.com/docs/>
10. Goodfellow, I., Bengio, Y., & Courville, A. (2016). Deep Learning. MIT Press.
11. Geron, A. (2019). Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow. O'Reilly Media.
12. IEEE Xplore — Research Papers on Network Intrusion Detection Systems. <https://ieeexplore.ieee.org/>